Development Build Log: Security Engine Construction

1. The Core Idea

The goal was to build a "Swiss Army Knife" for security scanning. Instead of relying on a single tool, we wanted a platform that could see a project from multiple angles: the code, the libraries, the secrets, and the infrastructure.

2. Tool Selection & Rationale

Layer	Options Considered	Chosen Tool	Reason for Choice
Application (SAST)	Semgrep, SonarQube	**Opengrep**	Switched from Semgrep to Opengrep (open-source fork) to ensure 100% license freedom and access to all advanced rules without paywalls.
Dependency (SCA)	Snyk, Safety, Trivy	**Trivy**	Trivy is fast, highly accurate, and supports almost all modern languages (Node, Python, Go, etc.) in a single binary.
Secrets	Trufflehog, Gitleaks	**Gitleaks**	Gitleaks is the industry standard for scanning git history and has the lowest false-positive rate for API keys.
Infra (Docker)	Hadolint, Terrascan	**Hadolint**	Lightweight and specifically designed to catch "security smells" in Dockerfiles.

3. Step-by-Step Development Process

Phase 1: Containerization (The Sandbox)

- **Challenge:** How to run untrusted code safely?
- **Solution:** We built `security-engine.Dockerfile`. This image contains all 4 tools pre-installed on a Debian-slim base.
- **Learning:** We encountered "PATH" issues where tools installed via scripts weren't found. We solved this by manually moving binaries to `/usr/local/bin` during the build process.

Phase 2: Manual Data Analysis

- **Goal:** Understand how tools "talk."
- **Action:** We cloned "vulnerable-by-design" repos (`NodeGoat`, `nodejs-goof`) and ran tools manually.
- **Discovery:** Every tool has a different JSON dialect. For example, Opengrep uses `extra.severity`, while Trivy uses a nested `Results[].Vulnerabilities[].Severity`.

Phase 3: The Normalization Layer

- **Action:** Developed `normalize.py` (V1 to V4).
- **Major Breakthrough (V2):** Discovered that Trivy scans multiple files (targets) and our script was only capturing the first one. We refactored to loop through all targets.
- **Major Breakthrough (V4):** Realized that SAST tools don't provide CVSS scores. We implemented a **Heuristic Fallback** system to map Severity (High/Med/Low) to estimated CVSS scores (8.0/5.5/2.5) so the Risk Engine (Bhumika's part) wouldn't receive 0.0 values.

Phase 4: Full Automation (The Orchestrator)

- **Action:** Developed `engine.py`.
- **Implementation:** Used Python's `threading` module to launch all 4 scans simultaneously.
- **Windows Fixes:** Solved the `PermissionError` during cleanup by adding a custom handler (`remove_readonly`) to force-delete git objects on Windows.

4. Key Architectural Decisions

1. **Isolated Workspaces:** Every scan gets a unique UUID and its own folder. This allows the engine to handle multiple users at once without data corruption.
2. **Unified Intelligence Schema:** We defined a "Contract" for the team. No matter which tool finds an issue, the final output always has the same fields: `id`, `tool`, `cvss`, `evidence`, and `fix`.
3. **Black Box Interface:** The entire engine is triggered by a single function: `orchestrate_scan(url)`.

5. Empirical Testing & Results Analysis

To verify the engine's effectiveness, we conducted rigorous testing against four industry-standard "Vulnerable-by-Design" repositories. This allowed us to verify that all four scanners (Opengrep, Trivy, Gitleaks, Hadolint) were correctly capturing data.

🧪 Test Subjects (Targets)

1. **NodeGoat (Node.js):** Used for initial manual testing. Proven to contain hardcoded secrets and SQLi.
2. **Nodejs-Goof (Node.js/Snyk):** A highly insecure repo used to stress-test the SCA (Trivy) and Docker (Hadolint) engines.
3. **DVNA (Damn Vulnerable Node App):** Used to verify the **Master Orchestrator** automation.
4. **PyGoat (Python/Django):** Used to prove the engine is **Language Agnostic**.

📂 Result Folders: What do they contain?

The project root contains two primary folders with JSON data (initial manual test results were purged after calibration). Here is what they represent:

1. `scan-results-goof/` (The Normalization V2 Benchmark)

Contains the results of scanning the `nodejs-goof` repo.

- **Key Finding:** `trivy.json` here is massive (~2MB), containing **183 library vulnerabilities**.
- **Key Finding:** `unified_report_v2.json` represents our first successful merge of 200+ diverse findings into a single schema.

2. `scan-results-automated/` (The Production Standard)

Contains the reports generated by the final `engine.py` script. These files use UUIDs (e.g., `82e9e959...`) to ensure no data is overwritten.

- **Case Study: PyGoat Scan (`82e9e959...`):**
  - **Opengrep:** Identified 16 Python-specific injection flaws.
  - **Gitleaks:** Discovered **10 hardcoded secrets**, including JWT tokens and Django CSRF tokens.
  - **Trivy:** Mapped out 200+ CVEs in the Python `requirements.txt`.

🔍 Key Intelligence Captured

Teammates can inspect these JSON files to see the "Raw vs Normalized" data:

- **Evidence Field:** Captures the exact line of code (e.g., `eyJ0eXAiOiJKV1Qi...`) for the AI Agent to explain.

- **CVSS Coverage:** Every entry in the `report_...json` files now has a 0-10.0 score, ensuring the **Risk Engine (Bhumika)** can calculate the final 0-100 project heat score immediately.

6. Verified Milestones

- ☑ Successfully scanned **Node.js** projects (DVNA, NodeGoat).
- ☑ Successfully scanned **Python** projects (PyGoat).
- ☑ Confirmed 100% capture of Secrets, SAST, and SCA findings.
- ☑ Automated cleanup of workspace after scan completion.
- ☑ Implemented Heuristic Fallback for 100% CVSS score coverage.